

Day 10

8 bit ALU:

Verilog Code:

```
module alu #(parameter WIDTH = 8)(
    input [WIDTH-1:0] a, b,
    input [3:0] opcode,
    output reg [WIDTH-1:0] result,
    output zero,
    output carry,
    output negative,
    output overflow
);

    reg carry_out;

    // Opcode definitions
    localparam ADD = 4'b0000;
    localparam SUB = 4'b0001;
    localparam AND = 4'b0010;
    localparam OR = 4'b0011;
    localparam XOR = 4'b0100;
    localparam NOT = 4'b0101;
    localparam SHL = 4'b0110; // shift left
    localparam SHR = 4'b0111; // shift right
    localparam SLT = 4'b1000; // set less than (signed)
    localparam SLTU = 4'b1001; // set less than unsigned
    localparam MUL = 4'b1010; // lower 8 bits of multiply
    localparam XNOR = 4'b1011;
    localparam NAND = 4'b1100;
    localparam NOR = 4'b1101;
    localparam PASS = 4'b1110; // pass A through
    localparam CLR = 4'b1111; // clear result

    always @(*) begin
        carry_out = 0;
        case(opcode)
            ADD: {carry_out, result} = a + b;
            SUB: {carry_out, result} = a - b;
            AND: result = a & b;
            OR: result = a | b;
            XOR: result = a ^ b;
            NOT: result = ~a;
            SHL: {carry_out, result} = {a, 1'b0}; // MSB into carry
            SHR: {result, carry_out} = {1'b0, a}; // LSB into carry
            SLT: result = ($signed(a) < $signed(b)) ? 1 : 0;
            SLTU: result = (a < b) ? 1 : 0;
            MUL: result = a * b;
            XNOR: result = ~(a ^ b);
            NAND: result = ~(a & b);
            NOR: result = ~(a | b);
            PASS: result = a;
            CLR: result = 0;
            default: result = 0;
        endcase
    end
```

```
// Flags
assign zero    = (result == 0);
assign carry   = carry_out;
assign negative = result[WIDTH-1];
assign overflow = (opcode == ADD) ?
    (~a[WIDTH-1] & ~b[WIDTH-1] & result[WIDTH-1]) |
    ( a[WIDTH-1] & b[WIDTH-1] & ~result[WIDTH-1]) :
    (opcode == SUB) ?
    (~a[WIDTH-1] & b[WIDTH-1] & result[WIDTH-1]) |
    ( a[WIDTH-1] & ~b[WIDTH-1] & ~result[WIDTH-1]) : 0;
Endmodule
```

Testbench:

```
`timescale 1ns/1ps
module tb;
    reg [7:0] a, b;
    reg [3:0] opcode;
    wire [7:0] result;
    wire    zero, carry, negative, overflow;

    integer pass, fail;

    alu uut(.a(a), .b(b), .opcode(opcode),
        .result(result), .zero(zero),
        .carry(carry), .negative(negative),
        .overflow(overflow));

    task check(
        input [7:0] exp_result,
        input    exp_zero,
        input    exp_carry,
        input [23:0] op_name
    );
        #10;
        if(result === exp_result && zero === exp_zero && carry === exp_carry) begin
            $display("PASS [%s] a=%h b=%h result=%h zero=%b carry=%b",
                op_name, a, b, result, zero, carry);
            pass = pass + 1;
        end
        else begin
            $display("FAIL [%s] a=%h b=%h | got result=%h zero=%b carry=%b | exp result=%h zero=%b carry=%b",
                op_name, a, b, result, zero, carry, exp_result, exp_zero, exp_carry);
            fail = fail + 1;
        end
    endtask

    initial begin
        $dumpfile("alu.vcd");
        $dumpvars(0, tb);
        pass = 0; fail = 0;

        // ADD
        a = 8'h0F; b = 8'h01; opcode = 4'b0000;
        check(8'h10, 0, 0, "ADD ");

        // ADD with carry
        a = 8'hFF; b = 8'h01; opcode = 4'b0000;
```

```

check(8'h00, 1, 1, "ADDc");

// SUB
a = 8'h0F; b = 8'h05; opcode = 4'b0001;
check(8'h0A, 0, 0, "SUB ");

// SUB zero result
a = 8'hAA; b = 8'hAA; opcode = 4'b0001;
check(8'h00, 1, 0, "SUBz");

// AND
a = 8'hF0; b = 8'hAA; opcode = 4'b0010;
check(8'hA0, 0, 0, "AND ");

// OR
a = 8'hF0; b = 8'h0F; opcode = 4'b0011;
check(8'hFF, 0, 0, "OR ");

// XOR
a = 8'hFF; b = 8'hFF; opcode = 4'b0100;
check(8'h00, 1, 0, "XOR ");

// NOT
a = 8'h55; b = 8'h00; opcode = 4'b0101;
check(8'hAA, 0, 0, "NOT ");

// SHL
a = 8'h01; b = 8'h00; opcode = 4'b0110;
check(8'h02, 0, 0, "SHL ");

// SHL with carry out
a = 8'h80; b = 8'h00; opcode = 4'b0110;
check(8'h00, 1, 1, "SHLc");

// SHR
a = 8'h80; b = 8'h00; opcode = 4'b0111;
check(8'h40, 0, 0, "SHR ");

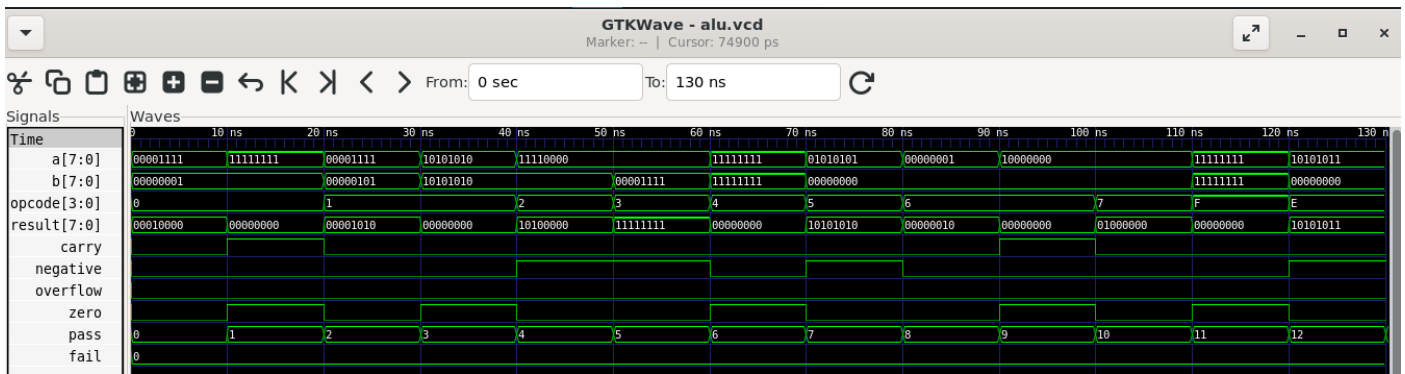
// CLR
a = 8'hFF; b = 8'hFF; opcode = 4'b1111;
check(8'h00, 1, 0, "CLR ");

// PASS
a = 8'hAB; b = 8'h00; opcode = 4'b1110;
check(8'hAB, 0, 0, "PASS");

$display("\n=== Results: %0d PASS, %0d FAIL ===", pass, fail);
$finish;
end

endmodule

```

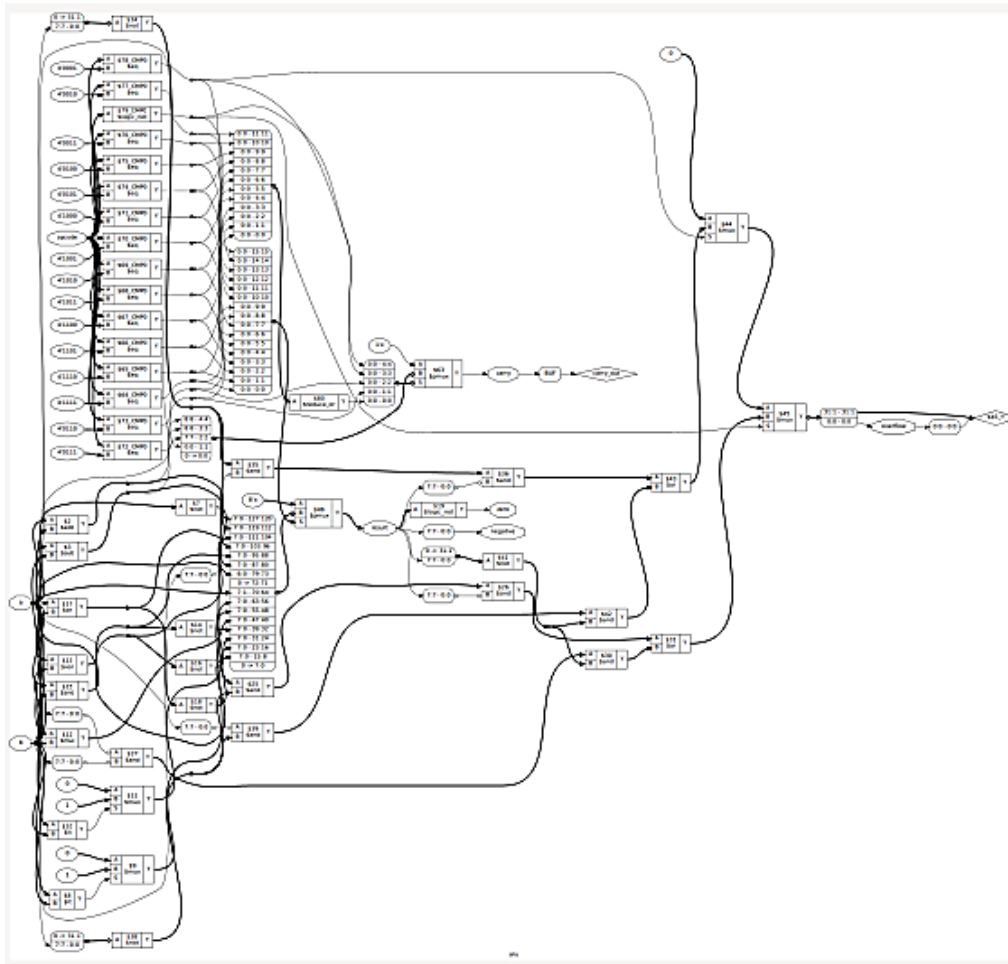


Simulation and Synthesis output:

VCD info: dumpfile alu.vcd opened for output.

PASS [ADD] a=0f b=01 result=10 zero=0 carry=0
 PASS [ADDc] a=ff b=01 result=00 zero=1 carry=1
 PASS [SUB] a=0f b=05 result=0a zero=0 carry=0
 PASS [SUBz] a=aa b=aa result=00 zero=1 carry=0
 PASS [AND] a=f0 b=aa result=a0 zero=0 carry=0
 PASS [OR] a=f0 b=0f result=ff zero=0 carry=0
 PASS [XOR] a=ff b=ff result=00 zero=1 carry=0
 PASS [NOT] a=55 b=00 result=aa zero=0 carry=0
 PASS [SHL] a=01 b=00 result=02 zero=0 carry=0
 PASS [SHLc] a=80 b=00 result=00 zero=1 carry=1
 PASS [SHR] a=80 b=00 result=40 zero=0 carry=0
 PASS [CLR] a=ff b=ff result=00 zero=1 carry=0
 PASS [PASS] a=ab b=00 result=ab zero=0 carry=0

=== Results: 13 PASS, 0 FAIL ===



2.26. Printing statistics.

=== alu ===

Number of wires:	55
Number of wire bits:	678
Number of public wires:	9
Number of public wire bits:	33
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	49
\$add	1
\$and	9
\$eq	15
\$logic_not	2
\$lt	2
\$mul	1
\$mux	4
\$not	7
\$or	3
\$pmux	2
\$reduce_or	1
\$sub	1
\$xor	1

Using if-else logic instead of case:

Verilog code:

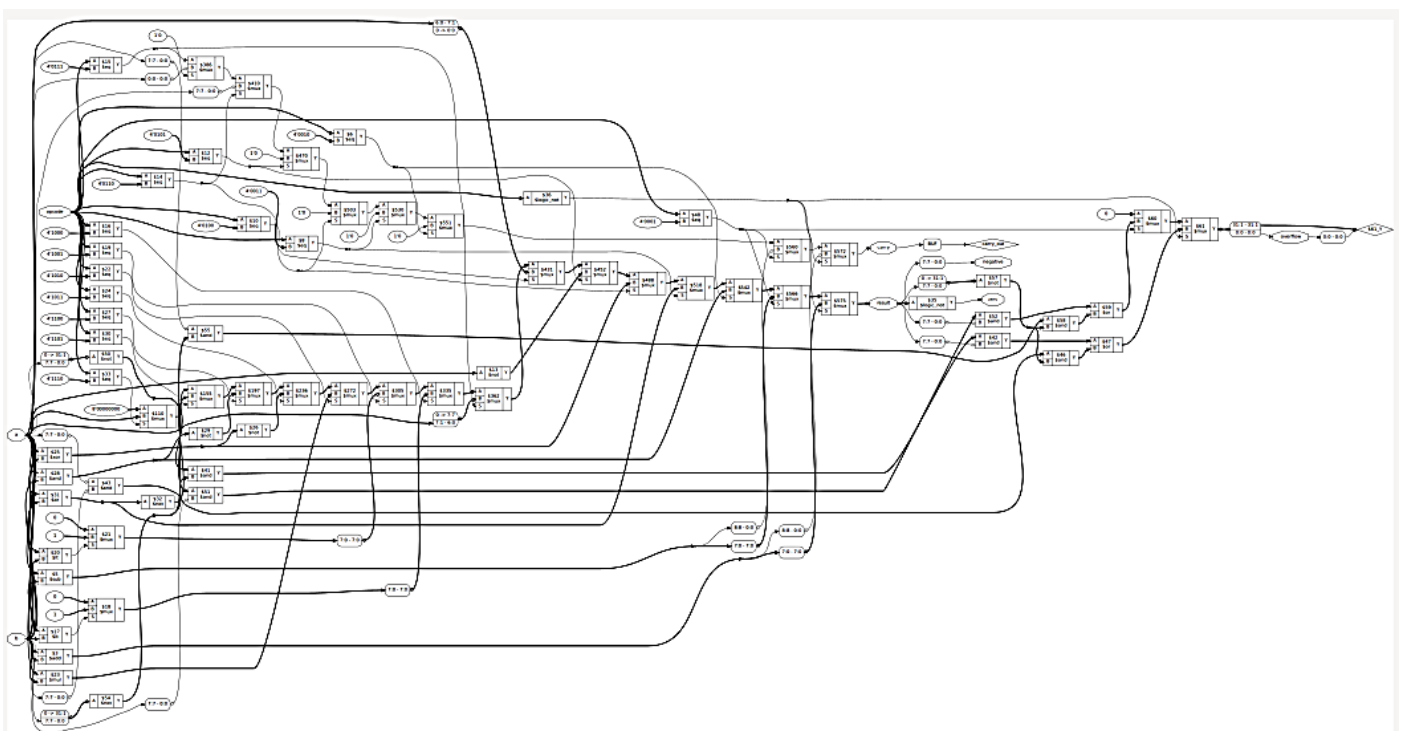
```
module alu #(parameter WIDTH = 8)(
    input [WIDTH-1:0] a, b,
    input [3:0] opcode,
    output reg [WIDTH-1:0] result,
    output zero,
    output carry,
    output negative,
    output overflow
);
    reg carry_out;

    // Opcode definitions
    localparam ADD = 4'b0000;
    localparam SUB = 4'b0001;
    localparam AND = 4'b0010;
    localparam OR = 4'b0011;
    localparam XOR = 4'b0100;
    localparam NOT = 4'b0101;
    localparam SHL = 4'b0110; // shift left
    localparam SHR = 4'b0111; // shift right
    localparam SLT = 4'b1000; // set less than (signed)
    localparam SLTU = 4'b1001; // set less than unsigned
    localparam MUL = 4'b1010; // lower 8 bits of multiply
    localparam XNOR = 4'b1011;
    localparam NAND = 4'b1100;
    localparam NOR = 4'b1101;
```

```
localparam PASS = 4'b1110; // pass A through
localparam CLR = 4'b1111; // clear result
```

```
always @(*) begin
    carry_out = 0;
    if (opcode == ADD) {carry_out, result} = a + b;
    else if(opcode == SUB) {carry_out, result} = a - b;
    else if(opcode == AND) result = a & b;
    else if(opcode == OR) result = a | b;
    else if(opcode == XOR) result = a ^ b;
    else if(opcode == NOT) result = ~a;
    else if(opcode == SHL) {carry_out, result} = {a, 1'b0}; // MSB into carry
    else if(opcode == SHR) {result, carry_out} = {1'b0, a}; // LSB into carry
    else if(opcode == SLT) result = ($signed(a) < $signed(b)) ? 1 : 0;
    else if(opcode == SLTU) result = (a < b) ? 1 : 0;
    else if(opcode == MUL) result = a * b;
    else if(opcode == XNOR) result = ~(a ^ b);
    else if(opcode == NAND) result = ~(a & b);
    else if(opcode == NOR) result = ~(a | b);
    else if(opcode == PASS) result = a;
    else if(opcode == CLR) result = 0;
    else result = 0;
end
```

```
// Flags
assign zero    = (result == 0);
assign carry   = carry_out;
assign negative = result[WIDTH-1];
assign overflow = (opcode == ADD) ?
    (~a[WIDTH-1] & ~b[WIDTH-1] & result[WIDTH-1]) |
    (a[WIDTH-1] & b[WIDTH-1] & ~result[WIDTH-1]) :
    (opcode == SUB) ?
    (~a[WIDTH-1] & b[WIDTH-1] & result[WIDTH-1]) |
    (a[WIDTH-1] & ~b[WIDTH-1] & ~result[WIDTH-1]) : 0;
endmodule
```



6. Printing statistics.

=== alu ===

Number of wires:	74
Number of wire bits:	795
Number of public wires:	9
Number of public wire bits:	33
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	68
\$add	1
\$and	9
\$eq	14
\$logic_not	2
\$lt	2
\$mul	1
\$mux	27
\$not	7
\$or	3
\$sub	1
\$xor	1

We can see that using if-else logic here has taken more number of cells than the case logic, since it is sequential there is a much higher delay too.